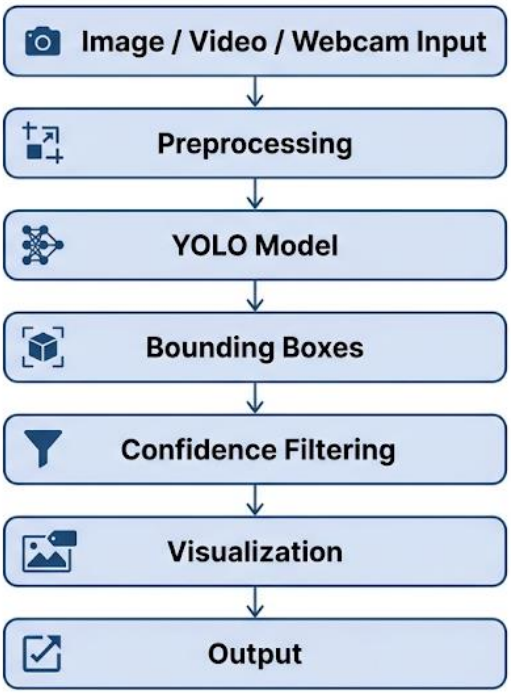
System Architecture Documentation

Hand Gesture Detection System | AI Summer Fellowship 2026, Computer Vision Track, Week 2, Assignment 8

1. Overview

The hand gesture detection system follows a linear, single-pass inference pipeline built around a custom-trained YOLOv8n model. A frame enters the pipeline as raw image data, is prepared for the model, passed through detection, filtered for confidence, visually annotated, and returned to the user as an output — either as a live webcam overlay or a saved image/video. The diagram below shows this end-to-end flow.

2. Architecture Diagram



Note: this is a text-based placeholder rendering of the pipeline for this document. A designed graphical version can be generated using the diagram prompt provided separately, and swapped into this section before final submission.

3. Pipeline Stage Details

Stage	What Happens	Tech Used
Input	User supplies a webcam frame, uploaded image, or video frame as raw pixel data (BGR array).	OpenCV, browser MediaStream API
Preprocessing	Frame is resized to the model's input resolution (640x640), normalized to [0,1], and reordered into the tensor format YOLO expects.	OpenCV, Ultralytics internal preprocessing
YOLO Model	The preprocessed tensor passes through the trained YOLOv8n network (backbone -> neck -> detection head) in a single forward pass, producing raw predictions for every anchor point.	YOLOv8n (Ultralytics), PyTorch

Stage	What Happens	Tech Used
Bounding Boxes	Raw predictions are decoded into candidate bounding boxes, each with x/y/width/height, an objectness score, and per-class probabilities. Non-Maximum Suppression (NMS) removes duplicate/overlapping boxes for the same object.	Ultralytics postprocessing, NMS
Confidence Filtering	Boxes below the confidence threshold (default 0.4, adjustable via the app's slider) are discarded, keeping only detections the model is sufficiently confident about.	Confidence threshold slider (app UI)
Visualization	Surviving boxes are drawn on the original frame with class label, confidence score, and a color-coded box per gesture class; a live legend panel and FPS counter are overlaid.	OpenCV drawing functions, HTML5 Canvas / JS
Output	The final annotated frame is streamed back to the browser in real time (webcam) or returned as a saved annotated image/video for export.	Flask response, browser rendering

4. Data Flow Summary

- Input arrives as a raw BGR frame (webcam, image upload, or video frame).
- Preprocessing standardizes every frame to a fixed 640x640 tensor regardless of source.
- The YOLO model performs one forward pass per frame — no separate region-proposal stage, which is what enables real-time speed.
- Bounding box decoding + NMS reduces raw model output to a clean, non-overlapping set of candidate detections.
- Confidence filtering is the only user-adjustable stage in the pipeline (via the UI slider), letting the user trade off precision vs. recall live.
- Visualization and output are presentation-layer only — they do not affect detection results, only how those results are displayed or saved.

5. Design Notes

This pipeline is intentionally linear and stateless per frame: each frame is processed independently with no memory of previous frames, which keeps latency low and avoids compounding errors across a video stream. The only tunable parameter exposed to the end user is the confidence threshold — all other stages (preprocessing, model architecture, NMS) are fixed at inference time, keeping the system predictable and easy to reason about during the technical interview portion of the evaluation.